

Sprawozdanie z projektu

Symulator Układu Automatycznej Regulacji (UAR)

Sekcja 3 (3-osobowa) · Politechnika Śląska

1. Skład sekcji i podział obowiązków

Sekcja 3-osobowa. Podział obowiązków w realizacji projektu (część nieoceniana):

Członek sekcji	Zakres obowiązków
Konrad Konieczny	Code review przejętej aplikacji oraz warstwa komunikacji sieciowej - klasa Sieci: serwer/klient TCP (QTcpServer/QTcpSocket), projekt ramki protokołu, buforowanie i składanie ramek nad strumieniem TCP. Serializacja próbek za pomocą QDataStream/JSON oraz przesyłanie konfiguracji UAR w formacie JSON.
Kacper Banasik	Korekta aplikacji, GUI i integracja z siecią - uporządkowanie timerów i pętli UAR, nawiązywanie i zrywanie połączenia, sygnalizacja stanu, blokowanie kontrolek, automatyczne przesyłanie konfiguracji JSON, obsługa zerwania połączenia oraz testy na localhost, w LAN i w połączeniu bezpośrednim.
Jakub Siwy	Krytyczne funkcjonalności sieciowe i logika symulacji rozproszonej w MainWindow - taktowanie jednostronne i obustronne, numeracja próbek (seq), wykrywanie opóźnień i braku nowych danych, podtrzymanie ostatniej wartości (ZOH), lampka wydajności oraz licznik pakietów na sekundę.

Podział jest umowny — wszystkie krytyczne funkcjonalności sieciowe były projektowane, testowane i poprawiane wspólnie podczas spotkań online.

2. Protokół komunikacyjny, serializacja i wykrywanie gubienia pakietów

2.1. Architektura połączenia

Regulator pełni rolę serwera i nasłuchuje na porcie (domyślnie 45763), a instancja obiektu jest klientem, który się z nim łączy. Łączność działa na localhost, w sieci LAN z serwerem DHCP oraz w połączeniu bezpośrednim komputer–komputer. Do transmisji używamy gniazd TCP (QTcpSocket, QTcpServer) z włączoną opcją małego opóźnienia oraz podtrzymania połączenia, co pozwala szybko wykryć zerwanie łącza.

2.2. Budowa ramki protokołu

Ponieważ TCP dostarcza ciągły strumień bajtów, a nie osobne wiadomości, zastosowaliśmy własny sposób wydzielania ramek. Każda ramka składa się z nagłówka o stałym rozmiarze oraz payloadu o zmiennej długości. Nagłówek zawiera jeden bajt określający typ pakietu (1 - konfiguracja, 2 - próbka) oraz czterobajtową długość payloadu. Po stronie odbiorczej dane trafiają do bufora, a dekodowanie rozpoczyna się dopiero po zgromadzeniu kompletnego nagłówka i całego payloadu. Dzięki temu poprawnie obsługujemy zarówno sklejone, jak i podzielone fragmenty strumienia TCP.

2.3. Typy przesyłanych danych

Na poziomie ramki stosujemy dwa typy pakietów: PAKIET_KONFIG oraz PAKIET_PROBKA. Pakiet konfiguracyjny przenosi pełne ustawienia UAR. W payloadzie pakietu próbki pole „rodzaj” rozróżnia sterowanie (regulator wysyła do obiektu sygnał u wraz z danymi PID), wyjście (obiekt odsyła wartość regulowaną y) oraz komendę start/stop/reset. Każda próbka zawiera również numer kolejny seq, czas symulacji, interwał i dane diagnostyczne potrzebne do oceny synchronizacji.

2.4. Serializacja

Konfiguracja UAR - parametry modelu ARX, regulatora PID, generatora, interwał i tryb taktowania - jest budowana jako QJsonObject i przesyłana w czytelnej formie JSON. Próbkę symulacji mogą być serializowane binarnie za pomocą QDataStream albo tekstowo w JSON; domyślnie używany jest szybszy format binarny. Wybór sposobu serializacji próbek odbywa się w GUI, a licznik pakietów na sekundę pozwala porównać wydajność obu wariantów.

2.5. Wykrywanie gubienia i opóźnień pakietów

TCP zapewnia uporządkowane dostarczenie danych, lecz przy zbyt małym interwale nowe próbki mogą docierać po terminie. Każda próbka ma numer kolejny seq i znacznik czasu, dlatego aplikacja wyznacza różnicę kroków oraz opóźnienie sieci. W trybie jednostronnym, gdy regulator nie otrzymał jeszcze nowej odpowiedzi obiektu, kontynuuje obliczenia na ostatniej wartości y (podtrzymanie ZOH). W trybie obustronnym obiekt analogicznie podtrzymuje poprzednie sterowanie u . Program sygnalizuje brak nowych danych, a zielona lub czerwona lampka - wraz z licznikiem pakietów na sekundę - informuje, czy transmisja wyrabia się w zadanym interwale. Spóźnione lub nieaktualne próbki nie cofają przebiegu symulacji w czasie.

3. Historia rozwoju funkcjonalności sieciowych

Prace prowadziliśmy etapami wyznaczonymi przez kolejne spotkania projektowe:

1. Spotkanie organizacyjne — omówienie regulaminu i podział pracy

- zapoznanie się z regulaminem projektu oraz wymaganiami tematu 1,
- ustalenie zakresu funkcjonalności sieciowych do zrealizowania przez sekcję 3-osobową,
- podział obowiązków między członków sekcji.

2. I spotkanie seminaryjne — code-review, wstępny projekt protokołu i serializacja

- przejście aplikacji lokalnej (ARX, PID, generator, pętla UAR) od poprzedniej sekcji,
- code review oraz weryfikacja aplikacji; wykrycie m.in. błędnego wskaźnika wykresu, stałego kroku $dt = 1$ s, nieprawidłowego resetu regulatora i powielonej logiki pętli UAR,
- opracowanie wstępnego projektu protokołu (nagłówek: typ i długość, payload z danymi),
- opracowanie klasy Sieci (serwer/klient TCP) oraz pierwsze przesłanie konfiguracji ARX/PID w formacie JSON między dwoma stanowiskami.

3. II spotkanie seminaryjne — korekta aplikacji i przesyłanie konfiguracji

- usunięcie dwóch niezależnych timerów, które powodowały wykonywanie kilku kroków obliczeniowych naraz i zaburzały pracę regulatora PID,
- połączenie powielonej logiki klas ProstyUAR i Petla_Sprzezenia oraz przeniesienie timera do warstwy obliczeniowej, aby oddzielić symulację od GUI,
- implementacja wyboru trybu regulatora lub obiektu, ręcznego rozłączania oraz automatycznej reakcji na nawiązanie i zerwanie połączenia,
- dodanie czytelnej sygnalizacji stanu połączenia i blokowania kontrolek zgodnie z aktywnym trybem pracy,
- automatyczne przesyłanie pełnej konfiguracji UAR w formacie JSON przy każdej zmianie ustawień, z zabezpieczeniem przed pętlą zwrotną.

4. III spotkanie seminaryjne — krytyczne funkcjonalności sieciowe

- uruchomienie symulacji UAR w sieci w trybie jednostronnym (jedna próbka wyjścia za jedną próbkę wejścia),
- dodanie trybu obustronnego z niezależnym taktowaniem obu instancji, sterowanego komendami z regulatora,
- umożliwienie wyboru binarnej (QDataStream) lub tekstowej (JSON) serializacji próbek oraz dodanie licznika pakietów na sekundę,
- implementacja wykrywania opóźnień i gubienia pakietów (seq, rozsynchronizowanie, ZOH, lampka wydajności).

5. IV spotkanie końcowe — walidacja aplikacji i zerowy termin odbiorów

- usunięcie pozostałych usterek w działaniu aplikacji,
- testy poprawności na localhost, w sieci LAN oraz w połączeniu bezpośrednim,
- obsługa zerwania połączenia z automatycznym powrotem do trybu stacjonarnego,
- demonstracja działającej symulacji rozproszonej na komputerach w sali.

4. Najistotniejsze napotkane trudności i sposoby ich rozwiązania

- Składanie wiadomości nad TCP - dane przychodzą jako strumień, dlatego fragmenty jednej ramki mogły być rozdzielone albo kilka ramek mogło zostać odebranych naraz.

Rozwiązaniem był nagłówek z typem i długością payloadu oraz buforowanie danych do chwili zebrania kompletnej wiadomości.

- Niespójne taktowanie symulacji - dwa timery oraz dwie klasy realizujące podobne obliczenia powodowały wielokrotne kroki i różne wyniki. Logikę scalono w jednej klasie, pozostawiono jeden timer, a krok Δt wyznaczany jest z rzeczywistego interwału.
- Opóźnienia przy małym interwale - nowa próbka nie zawsze docierała przed kolejnym taktem. Zastosowaliśmy numery seq, pomiar opóźnienia, podtrzymanie ostatniej wartości (ZOH) oraz lampkę wydajności, dzięki czemu symulacja zachowuje ciągłość, a problem jest widoczny dla operatora.
- Spójność konfiguracji i stanu GUI - ustawienia musiały automatycznie przechodzić między instancjami bez pętli zwrotnej, a niedostępne kontrolki powinny być blokowane. Konfigurację przesyłamy jako JSON, a stan kontrolki i wskaźnika połączenia jest aktualizowany po każdej zmianie trybu lub zerwaniu łącza.

5. Czego nauczył się każdy członek sekcji

Konrad Konieczny

- Zasady działania TCP oraz projektowania własnego protokołu aplikacyjnego z wydzielaniem ramek nad strumieniem bajtów.
- Serializacja danych w Qt - binarne próbki przez QDataStream oraz konfiguracja i wariant tekstowy próbek w formacie JSON.
- Obsługa gniazd i modelu serwer/klient w Qt (QTcpServer, QTcpSocket, sygnały i sloty).

Kacper Banasik

- Projektowanie GUI odpornego na błędy operatora: blokowanie kontrolki, sygnalizacja połączenia i ochrona przed przypadkową zmianą trybu.
- Zarządzanie stanem aplikacji podczas przełączania trybów sieciowy/stacjonarny oraz automatyczna synchronizacja konfiguracji JSON.
- Testowanie aplikacji sieciowej na localhost, w sieci LAN i w połączeniu bezpośrednim oraz obsługa sytuacji awaryjnych, takich jak zerwanie połączenia.

Jakub Siwy

- Synchronizacja symulacji rozproszonej oraz różnica między taktowaniem jednostronnym a obustronnym.
- Praktyczne radzenie sobie z opóźnieniami: podtrzymanie ostatniej wartości (ZOH), numeracja seq i śledzenie rozsynchronizowania.
- Projektowanie czytelnych wskaźników jakości transmisji: lampka „wyrabia się / nie wyrabia się” oraz licznik pakietów na sekundę.